

AI/ML Mini Project Report

Fake News Detector using BERT

Student Name	Vikky Kumar
College Name	Guru Jambheshwar University of Science and Technology, Hisar, Haryana
Branch & Year	Computer Science & Engineering (CSE), 3rd Year
Email	skvickyadav942@gmail.com
Phone Number	6287062072
Submission Date	30/05/2026

2. Project Overview

Problem Statement

Fake news refers to false or misleading information that is deliberately created and spread as if it were real news. It is increasingly difficult to distinguish between genuine and fabricated news articles, especially with the rapid growth of social media. The situation becomes especially dangerous during critical events such as elections or public health crises like pandemics, where misinformation can cause panic, influence public opinion, and even pose threats to human lives. There is a pressing need to detect and prevent the spread of fake news at an early stage before it reaches a large audience.

Objective

The main goal of this project is to build an automated Fake News Detector that can classify a given news article as either Real or Fake. The system uses the pre-trained BERT (Bidirectional Encoder Representations from Transformers) model, fine-tuned on a labeled news dataset, to perform binary text classification with high accuracy.

Technologies Used

- Python
- BERT (bert-base-uncased) — Pre-trained NLP Model
- PyTorch
- Hugging Face Transformers
- Pandas & NumPy
- Scikit-learn
- Matplotlib & Seaborn
- WordCloud

3. Dataset Details

Field	Details
Dataset Name	Fake News Dataset (news.csv)
Dataset Source	Kaggle
Total Records	6,335 rows
REAL News Records	3,171
FAKE News Records	3,164
Columns	Unnamed Index, Title, Text, Label
Label Values	REAL = 1, FAKE = 0
Dataset Link	https://www.kaggle.com/datasets/clmentbisailon/fake-and-real-news-dataset

4. Project Workflow

1. Data Collection

The dataset news.csv was obtained from Kaggle. It contains 6,335 labeled news articles with columns for title, text, and label (REAL or FAKE).

2. Data Preprocessing

The label column was converted from string values ('REAL', 'FAKE') to numeric values (1 and 0 respectively). Only the 'text' and 'label' columns were retained. Null values were dropped, and the dataset was shuffled randomly with a fixed seed for reproducibility.

3. Exploratory Data Analysis (EDA)

A bar chart was plotted to visualize the distribution of Fake vs Real news. A histogram was generated to compare the text length distribution of both categories. Word clouds were created separately for Fake and Real news to highlight the most frequently occurring words.

4. Data Splitting

The dataset was split into training (80%) and testing (20%) sets, resulting in 5,068 training samples and 1,267 test samples.

5. Tokenization

The pre-trained BERT tokenizer (bert-base-uncased) was used to tokenize all text samples with a maximum sequence length of 128 tokens, padding and truncation applied.

6. Dataset & DataLoader Creation

A custom PyTorch Dataset class (NewsDataset) was created to handle tokenization and return input tensors. DataLoaders were created with a batch size of 16.

7. Model Loading

The pre-trained BERT model (BertForSequenceClassification) was loaded with one additional output classification layer (num_labels=2). The model was moved to GPU (T4) for fast training.

8. Model Training

The model was fine-tuned for 3 epochs using the AdamW optimizer with a learning rate of $2e-5$. Training loss decreased from 0.2050 to 0.0328 across epochs.

9. Model Evaluation

The model was evaluated on the test set. Accuracy, precision, recall, and F1-score were computed. A confusion matrix heatmap was also plotted.

10. Result Prediction

A predict() function was implemented to classify any new input text as Real News or Fake News using the trained model.

5. Model / Algorithm Used

BERT — Bidirectional Encoder Representations from Transformers

BERT is a state-of-the-art pre-trained Natural Language Processing (NLP) model developed by Google. It is open-source and has been trained on a massive corpus of text data. BERT reads text in both directions (left-to-right and right-to-left) simultaneously, which allows it to understand the full context of a word within a sentence.

Why BERT was selected:

- BERT is pre-trained on large amounts of text data, which means it already understands the structure and semantics of the English language without training from scratch.
- It requires only one additional output layer for classification tasks, making it efficient and easy to fine-tune.
- BERT achieves state-of-the-art performance on text classification benchmarks.
- It handles long-range dependencies in text effectively, which is important for news articles.
- The model is open-source and freely available through the Hugging Face Transformers library.

Model Configuration:

Parameter	Value
Model Name	bert-base-uncased
Number of Labels	2 (FAKE = 0, REAL = 1)
Max Token Length	128
Optimizer	AdamW
Learning Rate	2e-5
Batch Size	16
Epochs	3
Device	CUDA (T4 GPU)

6. Implementation

Important Code Snippets

Data Loading and Preprocessing

```
df = pd.read_csv("news.csv")
df["label"] = df["label"].str.upper().str.strip()
df["label"] = df["label"].map({"REAL": 1, "FAKE": 0})
df = df[["text", "label"]]
df = df.dropna()
df = df.sample(frac=1, random_state=42).reset_index(drop=True)
```

Custom Dataset Class

```
class NewsDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len=128):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __getitem__(self, idx):
        encoding = self.tokenizer(
            self.texts[idx], max_length=self.max_len,
            padding="max_length", truncation=True,
            return_tensors="pt"
        )
        return {
            "input_ids": encoding["input_ids"].squeeze(),
            "attention_mask": encoding["attention_mask"].squeeze(),
            "label": torch.tensor(self.labels[idx], dtype=torch.long)
        }
```

Loading BERT with Output Layer

```
model = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
```

Training Loop

```
EPOCHS = 3
for epoch in range(EPOCHS):
    model.train()
    total_loss = 0
    for batch in train_loader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["label"].to(device)
        optimizer.zero_grad()
        outputs = model(input_ids=input_ids,
                        attention_mask=attention_mask, labels=labels)
        loss = outputs.loss
        total_loss += loss.item()
    loss.backward()
```

```
optimizer.step()
avg_loss = total_loss / len(train_loader)
print(f"Epoch {epoch+1}/{EPOCHS} - Loss: {avg_loss:.4f}")
```

Prediction Function

```
def predict(text):
    model.eval()
    encoding = tokenizer(text, max_length=128,
        padding="max_length", truncation=True, return_tensors="pt")
    input_ids = encoding["input_ids"].to(device)
    attention_mask = encoding["attention_mask"].to(device)
    with torch.no_grad():
        outputs = model(input_ids=input_ids,
            attention_mask=attention_mask)
        pred = torch.argmax(outputs.logits, dim=1).item()
    return "Real News" if pred == 1 else "Fake News"
```

7. Results

Model Accuracy

The BERT model achieved an overall accuracy of 95.03% on the test set of 1,267 samples.

Training Loss per Epoch

Epoch	Average Training Loss
Epoch 1	0.2050
Epoch 2	0.0667
Epoch 3	0.0328

Classification Report

Class	Precision	Recall	F1-Score	Support
Fake News (0)	0.93	0.98	0.95	655
Real News (1)	0.97	0.92	0.95	612
Accuracy			0.95	1267
Macro Avg	0.95	0.95	0.95	1267
Weighted Avg	0.95	0.95	0.95	1267

Confusion Matrix

The confusion matrix below shows the number of correctly and incorrectly classified samples for both classes.

	Predicted: Fake	Predicted: Real
Actual: Fake	642 (True Negative)	13 (False Positive)
Actual: Real	50 (False Negative)	562 (True Positive)

Output Predictions

Input Text	Predicted Output
Bill Gates is putting microchips in COVID vaccines to control humans.	Fake News
The Federal Reserve raised interest rates by 0.25 percent on Wednesday.	Real News
NASA confirms that the Earth is actually flat and has been hiding it for years.	Fake News

8. Challenges Faced

- **Slow Training on CPU:**

The BERT model has over 110 million parameters, which made training extremely slow on a local CPU. The training loop was taking over 50 minutes per epoch without any progress. This was resolved by switching to Google Colab and enabling the free T4 GPU.

- **Dataset Format Mismatch:**

The original project template assumed separate True.csv and Fake.csv files. However, the available dataset was a single combined file (news.csv) with a label column containing string values 'REAL' and 'FAKE'. Custom preprocessing code was written to handle this format.

- **Incorrect Prediction (All Fake):**

Initially, the predict() function was returning 'Fake News' for every input regardless of content. This was identified as an issue in the function logic and was fixed by adding softmax probability output and correcting the argmax-based prediction logic.

- **Label Encoding:**

The label column had string values ('REAL', 'FAKE') that needed to be mapped to integers (1 and 0) for the model to process. Careful mapping and null value removal were required to ensure no data loss or mislabeling.

- **Understanding BERT Architecture:**

Integrating a pre-trained transformer model into a custom PyTorch training pipeline required understanding of tokenization, attention masks, and the sequence classification head added on top of the base BERT model.

9. Conclusion

What Was Achieved

This project successfully demonstrates the use of BERT, a pre-trained transformer-based NLP model, for fake news detection. The model was fine-tuned on a balanced dataset of 6,335 news articles and achieved an accuracy of 95.03% on the test set. The system is capable of correctly classifying news articles as Real or Fake with high precision and recall for both classes. Exploratory data analysis including bar charts, histograms, and word clouds provided meaningful insights into the dataset before model training.

Future Improvements

- Deploy the model as a web application using Streamlit or Flask so that users can input any news article and get real-time predictions.
- Extend the model to support multilingual fake news detection using multilingual BERT (mBERT).
- Incorporate additional features such as news source credibility and author information to improve detection accuracy.
- Use larger and more diverse datasets including social media posts to improve generalization.
- Experiment with other transformer models such as RoBERTa or DistilBERT for faster inference without significant loss in accuracy.

10. References

- **Dataset:**

Fake and Real News Dataset — Kaggle

<https://www.kaggle.com/datasets/clmentbisaillon/fake-and-real-news-dataset>

- **BERT Paper:**

Devlin et al., 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding', 2018 <https://arxiv.org/abs/1810.04805>

- **Hugging Face Transformers Documentation:**

<https://huggingface.co/docs/transformers>

- **PyTorch Documentation:**

<https://pytorch.org/docs/stable/index.html>

- **Scikit-learn Documentation:**

<https://scikit-learn.org/stable/documentation.html>

- **Google Colab:**

<https://colab.research.google.com>